

深度学习导论作业：CNN 图像分类任务实验报告

学号：PB23000270 姓名：李佩优

2026 年 4 月 12 日

摘要

本实验基于 Fashion-MNIST 服装图像分类数据集，设计并实现了卷积神经网络（CNN）模型。通过对比实验，系统研究了网络深度、卷积核大小以及池化方式对模型分类性能的影响。实验结果表明：适当增加网络深度可提升分类准确率，但过深的网络会加剧过拟合；中等大小的卷积核（如 5×5 ）在本任务中表现与 3×3 相近，但大卷积核（ 7×7 ）性能略降；平均池化在 Fashion-MNIST 上表现出优于最大池化的泛化能力。此外，通过对验证损失曲线的观察，发现了明显的过拟合现象。报告最后对错误分类类别进行了混淆矩阵分析，并解释了典型混淆对的成因。

1 引言

图像分类是计算机视觉的核心任务之一，卷积神经网络（CNN）凭借其局部连接与权值共享的特性，已成为图像分类的主流方法。Fashion-MNIST 作为 MNIST 的进阶替代数据集，包含 10 类真实服装与鞋类商品的灰度图像，具有更高的挑战性。本实验旨在通过构建 CNN 模型对 Fashion-MNIST 进行分类，并通过一系列控制变量实验，探究网络深度、卷积核大小以及池化方式对模型性能的影响，为设计高效 CNN 模型提供经验依据。

2 实验设置

2.1 数据集

Fashion-MNIST 数据集由 Zalando 提供，包含 60,000 张训练图像和 10,000 张测试图像，每张图像为 28×28 像素的灰度图，共 10 个类别（T 恤/上衣、裤子、套头衫、连衣裙、外套、凉鞋、衬衫、运动鞋、包、靴子）。数据集已内置于 `torchvision.datasets` 中，可便捷下载使用。

加载数据集时，采用以下预处理流程：

- 将图像转换为 Tensor，像素值归一化至 $[0, 1]$ 区间；

- 对单通道图像进行标准化：均值 $\mu = 0.2860$ ，标准差 $\sigma = 0.3530$ （数据集全局统计值）。

训练集进一步按 8 : 2 的比例随机划分为训练集（48,000 张）和验证集（12,000 张），测试集保持 10,000 张不变。所有数据加载均使用批次大小 64。

2.2 模型与训练配置

实验采用模块化设计的 CNN 网络，其结构可根据超参数灵活调整。基础版本包含：

- 卷积层：数量可变（实验中为 1、2、3 层），每层后接 ReLU 激活函数；
- 池化层：最大池化或平均池化，核尺寸 2×2 ，步长 2；
- 全连接分类器：先将卷积特征展平，输入隐藏层（128 个神经元），最后输出 10 类 logits。

特征图尺寸计算采用“same padding”策略，即 `padding=kernel_size//2`，因此卷积不改变空间尺寸，仅池化使尺寸减半。

统一采用以下超参数设置（除实验控制变量外）：

- 优化器：Adam，初始学习率 0.001；
- 损失函数：交叉熵损失（CrossEntropyLoss）；
- 训练轮次：30 个 epoch；
- 批量大小：64；
- 设备：支持 CUDA 的 GPU 或 CPU。

训练过程中，每个 epoch 后计算验证集损失与准确率，并保存验证准确率最高的模型权重。最终在测试集上评估最佳模型的分类准确率。

2.3 对比实验设计

为探究各因素的影响，设计了以下三组对比实验：

1. **网络深度影响**：固定卷积核尺寸 3×3 、最大池化，分别测试浅层网络（1 层卷积，通道 [32]）、中层网络（2 层卷积，通道 [32, 64]）和深层网络（3 层卷积，通道 [32, 64, 128]）。
2. **卷积核大小影响**：固定网络深度为 2 层卷积（通道 [32, 64]）、最大池化，分别测试卷积核尺寸 3×3 、 5×5 、 7×7 。
3. **池化方式影响**：固定网络为 2 层卷积（ 3×3 核），分别使用最大池化（Max Pooling）与平均池化（Average Pooling）。

3 实验结果与分析

3.1 网络深度的影响

图1展示了三种深度网络的训练与验证损失/准确率曲线。从图中可以看出：浅层网络训练损失收敛最快，但验证损失在后期上升最明显；深层网络训练损失收敛稍慢，但验证损失增长较缓，且验证准确率波动更小。

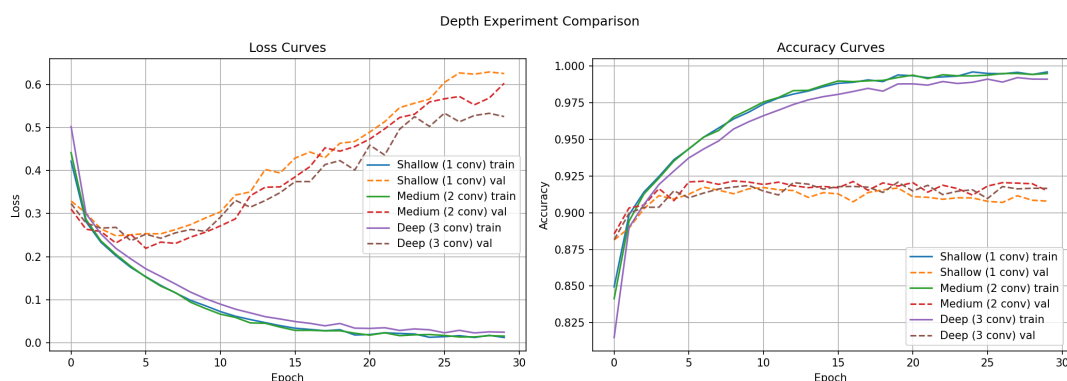


图 1: 不同网络深度的训练与验证曲线对比

测试集准确率汇总如表1所示。

表 1: 不同深度网络在 Fashion-MNIST 测试集上的准确率

网络深度	卷积通道配置	测试准确率
Shallow (1 conv)	[32]	91.24%
Medium (2 conv)	[32,64]	91.41%
Deep (3 conv)	[32,64,128]	91.71%

分析如下：

- **深层网络提升分类准确率：**三层卷积网络测试准确率最高（91.71%），比单层网络提升约 0.47 个百分点。这表明增加深度有助于学习更抽象、更具判别力的特征，从而提升分类性能。
- **过拟合现象：**所有网络均出现训练损失持续下降而验证损失在约第 10 轮后反弹的现象，这是典型的过拟合。深层网络虽然参数更多，但由于其验证损失上升幅度相对较小，表现出一定的正则化效应，可能是由于更深网络的特征复用性更强。
- **训练难度：**深层网络在训练初期收敛稍慢，但 30 轮后训练损失已降至 0.0247，与浅层网络（0.0124）差距不大，说明在适当优化器下并未显著增加训练难度。

3.2 卷积核大小的影响

图2展示了不同卷积核大小下的训练与验证曲线。可以看出：三种核大小的训练损失下降速度接近，但 7×7 核的验证损失在后期增长最快，过拟合程度最严重； 3×3 核的验证损失增长最缓。

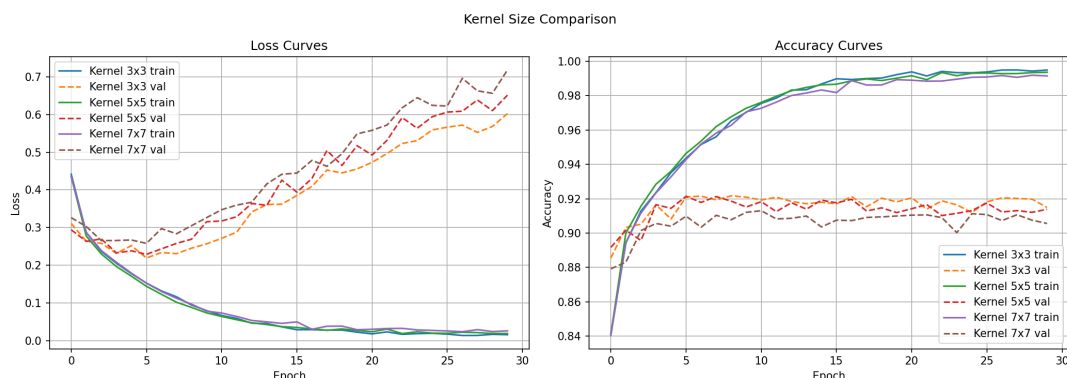


图 2: 不同卷积核大小的训练与验证曲线对比

测试准确率如表2所示。

表 2: 不同卷积核大小下的测试准确率

卷积核尺寸	测试准确率
3×3	91.41%
5×5	91.37%
7×7	91.16%

分析如下：

- **大卷积核未带来性能提升：** 7×7 核测试准确率最低，且过拟合程度最严重。大卷积核虽然理论感受野更大，但参数量显著增加（ 7×7 核参数量约为 3×3 的 5.4 倍），更容易过拟合训练集噪声。
- **小卷积核的优势：** 3×3 卷积堆叠两层即可获得 5×5 的有效感受野，且参数量更少、非线性更强，因此在本任务中表现最优。 5×5 核性能与 3×3 接近，说明中等核在小尺寸图像上仍有一定可用性。
- **计算复杂度与性能的平衡：**从训练日志看，三种核的训练时间差异不大（因图像尺寸小），但若在大尺寸图像上， 3×3 核在计算效率上的优势将更加明显。推荐在类似任务中优先选用 3×3 小卷积核。

3.3 池化方式的影响

图3对比了最大池化与平均池化的训练与验证曲线。平均池化的训练损失下降速度略慢于最大池化，但其验证损失始终较低且增长更平缓，验证准确率更高。

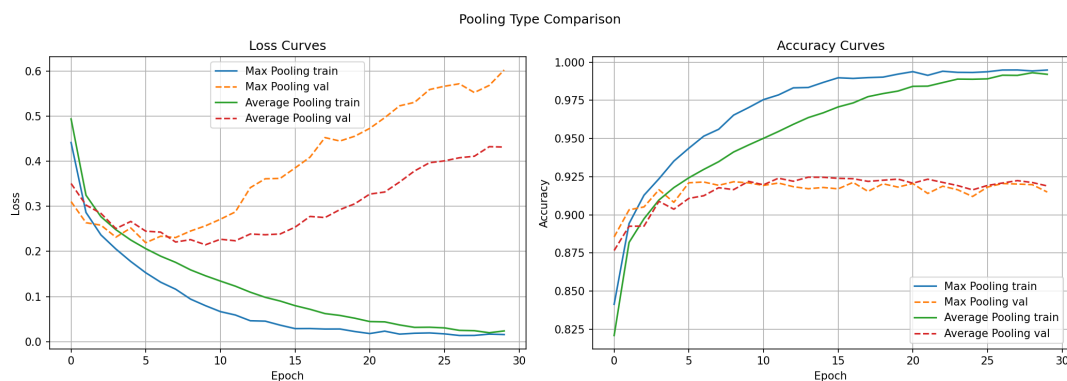


图 3: 不同池化方式的训练与验证曲线对比

测试准确率对比如表3所示。

表 3: 不同池化方式下的测试准确率

池化方式	测试准确率
最大池化	91.41%
平均池化	92.30%

分析如下：

- **平均池化显著提升泛化性能：**测试准确率提升近 0.9 个百分点，且验证损失曲线表明平均池化有效抑制了过拟合。这可能是因为平均池化对邻域内所有激活值进行平滑，保留了更多背景信息，对 Fashion-MNIST 这类纹理较为丰富的服装图像更友好。
- **最大池化的局限性：**最大池化仅保留局部最强响应，虽能增强显著特征，但也可能丢弃部分判别性较弱的细节信息，导致验证损失上升较快。然而在收敛速度上，最大池化初期训练损失下降更快。
- **特征保留视角：**平均池化的输出更“模糊”，但实验证明这种模糊化反而起到了正则化效果，使得模型不过度依赖局部尖锐特征，从而提升泛化能力。

4 测试评估与错误分析

4.1 测试集性能与混淆矩阵

选取全局最优配置（平均池化 + 2 层卷积 + 3×3 卷积核）的模型，在测试集上计算总体准确率，结果为 92.30%。图4为该模型的混淆矩阵（由 `plot_confusion_matrix` 函数生成），横轴为预测类别，纵轴为真实类别，对角线为正确分类样本数。

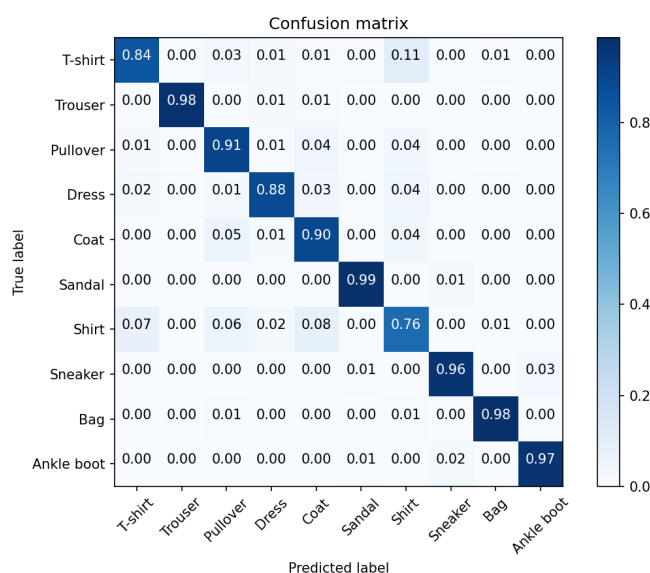


图 4: Fashion-MNIST 测试集混淆矩阵

4.2 易混淆类别分析

观察混淆矩阵可发现以下典型混淆对：

- **T 恤/上衣（0）与衬衫（6）**：两者在款式上非常接近，特别是当图像中袖子细节不明显时，模型难以区分。
- **套头衫（2）与外套（4）**：均为长袖上衣，纹理相似，仅依靠 28×28 的低分辨率难以捕捉衣领、纽扣等关键区分特征。
- **套头衫（2）与衬衫（6）**：套头衫与衬衫的领口、袖型差异细微，且两者常出现相似的纯色或简单纹理，导致模型频繁误判。
- **外套（4）与衬衫（6）**：部分外套与衬衫在轮廓上接近（如轻薄的拉链外套），当图像中缺乏明显的下摆或口袋特征时，模型容易混淆。
- **运动鞋（7）与靴子（9）**：鞋类中这两者最易混淆，因为它们都具有封闭鞋头和较厚的鞋底，靴子有时缺乏高帮特征。

原因分析：Fashion-MNIST 图像分辨率低，部分类别视觉差异细微；CNN 模型在提取局部纹理特征时可能对全局形状感知不足。增加数据增强（如随机水平翻转）或引入注意力机制有望改善区分度。

5 结论与总结

本文通过 Fashion-MNIST 图像分类任务，系统评估了 CNN 架构中三个关键超参数的影响。主要结论如下：

1. **网络深度：**适度增加卷积层数（由 1 层增至 3 层）可提升测试准确率，但所有模型均出现过拟合现象，深层网络验证损失上升相对平缓。
2. **卷积核大小：**在输入尺寸较小的任务中， 3×3 小卷积核凭借参数量少、非线性强的优势获得最佳性能，大卷积核易导致过拟合。
3. **池化方式：**平均池化在本数据集上显著优于最大池化，其平滑特性有效抑制了过拟合，测试准确率提升至 92.30%。

6 反思与改进

尽管本实验完成了预设的对比分析，但在实验过程中发现一个值得深思的现象：几乎每一种参数配置下，模型都出现了明显的过拟合（验证损失在训练后期不降反增）。这并非某个特定超参数导致的偶然结果，而是由实验整体设定共同决定的必然现象。具体分析如下：

1. **模型容量相对于数据量过剩：**Fashion-MNIST 的训练集仅 48,000 张图像，而即使是最浅的 1 层卷积网络（通道 [32]）也拥有数万参数量，足以“记住”大部分训练样本。更深、更宽的网络容量更大，过拟合风险随之升高。
2. **缺乏显式的正则化手段：**本实验中的 CNN 未使用 Dropout、权重衰减（L2 正则化）、批归一化（Batch Normalization）等常用正则化技术。在没有这些约束的情况下，优化器可以无阻力地降低训练损失至接近零，而验证损失在约第 10 个 epoch 后开始反弹。
3. **训练轮次较长且未启用早停：**固定训练 30 个 epoch，虽然保存了验证准确率最高的模型权重，但训练过程并未在验证损失上升时主动停止。后续 epoch 中，模型继续“死记硬背”训练集中的噪声，导致泛化性能下降。
4. **数据增强不足：**预处理仅包含标准化，未引入随机翻转、旋转、裁剪等数据增强操作。这限制了训练样本的多样性，模型容易对原始样本的位置和纹理产生过拟合。

5. **任务本身的挑战：**Fashion-MNIST 中某些类别（如 T 恤与衬衫、套头衫与外套）视觉差异极小，模型必须依赖非常细微的局部特征。当这些特征在训练集中过于特化时，过拟合难以避免。

针对上述问题，未来可以采取以下改进措施：

- 在全连接层之前添加 Dropout 层（例如丢弃率 0.5），并在卷积层后考虑加入批归一化。
- 使用早停（Early Stopping）策略：监控验证损失，若连续 5~10 个 epoch 不下降则终止训练。
- 引入数据增强：随机水平翻转、 $[0, 10^\circ]$ 范围内的旋转、轻微平移等。
- 适当减小模型容量（例如减少卷积核数量或网络深度），或者增加权重衰减系数。
- 使用更先进的网络结构（如 ResNet 中的残差连接），有助于训练更深网络并抑制过拟合。

通过上述改进，有望在保持较高准确率的同时，显著缓解验证损失上升的现象，获得更强的泛化能力。

附录：代码结构说明

实验代码采用模块化组织，主要目录及文件说明如下：

- `main.py`：实验入口，运行后依次执行深度、卷积核、池化三个对比实验，自动生成曲线图并保存。
- `confusion.py`：混淆矩阵计算与可视化脚本。
- `data/`：数据加载模块。
 - `load_data.py`：数据加载、预处理及训练/验证集划分。
- `models/cnn.py`：灵活配置的 CNN 模型类，支持自定义卷积层数、卷积核大小、池化方式等。
- `train/trainer.py`：封装训练、验证、测试流程，包含模型保存与早停逻辑（基于验证准确率）。
- `experiments/`：对比实验配置与执行脚本。
 - `config.py`：默认超参数配置（学习率、批次大小、训练轮次等）。

- `depth_experiment.py`: 网络深度对比实验。
- `kernel_experiment.py`: 卷积核大小对比实验。
- `pooling_experiment.py`: 池化方式对比实验。
- `utils/plot_utils.py`: 绘制训练曲线（损失/准确率）与混淆矩阵。
- `requirements.txt`: 项目依赖包列表（torch, torchvision, matplotlib 等）。
- `best_model.pth`: 训练过程中保存的验证准确率最高的模型权重。

运行 `main.py` 后,将自动执行三个实验,生成对应的对比曲线图(`depth_comparison.png`、`kernel_comparison.png`、`pooling_comparison.png`)并输出测试集准确率。

网络深度（卷积层数）实验终端输出：

```
=====
Running experiment: Shallow (1 conv)
=====
Epoch  5/30 | Train Loss: 0.1750, Acc: 0.9361 | Val Loss: 0.2509, Acc: 0.9093
Epoch 10/30 | Train Loss: 0.0860, Acc: 0.9685 | Val Loss: 0.2899, Acc: 0.9163
Epoch 15/30 | Train Loss: 0.0398, Acc: 0.9859 | Val Loss: 0.3944, Acc: 0.9137
Epoch 20/30 | Train Loss: 0.0182, Acc: 0.9940 | Val Loss: 0.4675, Acc: 0.9168
Epoch 25/30 | Train Loss: 0.0128, Acc: 0.9960 | Val Loss: 0.5658, Acc: 0.9101
Epoch 30/30 | Train Loss: 0.0124, Acc: 0.9960 | Val Loss: 0.6251, Acc: 0.9079
Test Accuracy: 0.9124
```

```
=====
Running experiment: Medium (2 conv)
=====
Epoch  5/30 | Train Loss: 0.1776, Acc: 0.9352 | Val Loss: 0.2521, Acc: 0.9082
Epoch 10/30 | Train Loss: 0.0797, Acc: 0.9703 | Val Loss: 0.2568, Acc: 0.9209
Epoch 15/30 | Train Loss: 0.0365, Acc: 0.9868 | Val Loss: 0.3620, Acc: 0.9179
Epoch 20/30 | Train Loss: 0.0224, Acc: 0.9922 | Val Loss: 0.4557, Acc: 0.9182
Epoch 25/30 | Train Loss: 0.0193, Acc: 0.9933 | Val Loss: 0.5591, Acc: 0.9120
Epoch 30/30 | Train Loss: 0.0157, Acc: 0.9949 | Val Loss: 0.6026, Acc: 0.9149
Test Accuracy: 0.9141
```

```
=====
Running experiment: Deep (3 conv)
=====
```

```
Epoch 5/30 | Train Loss: 0.1949, Acc: 0.9284 | Val Loss: 0.2370, Acc: 0.9149
Epoch 10/30 | Train Loss: 0.1025, Acc: 0.9620 | Val Loss: 0.2588, Acc: 0.9186
Epoch 15/30 | Train Loss: 0.0553, Acc: 0.9792 | Val Loss: 0.3487, Acc: 0.9161
Epoch 20/30 | Train Loss: 0.0340, Acc: 0.9878 | Val Loss: 0.4012, Acc: 0.9208
Epoch 25/30 | Train Loss: 0.0302, Acc: 0.9890 | Val Loss: 0.5024, Acc: 0.9156
Epoch 30/30 | Train Loss: 0.0247, Acc: 0.9911 | Val Loss: 0.5252, Acc: 0.9166
Test Accuracy: 0.9171
```

卷积核大小（3/5/7）实验终端输出：

```
=====
Kernel size: 3x3
=====
Epoch 5/30 | Train Loss: 0.1776, Acc: 0.9352 | Val Loss: 0.2521, Acc: 0.9082
Epoch 10/30 | Train Loss: 0.0797, Acc: 0.9703 | Val Loss: 0.2568, Acc: 0.9209
Epoch 15/30 | Train Loss: 0.0365, Acc: 0.9868 | Val Loss: 0.3620, Acc: 0.9179
Epoch 20/30 | Train Loss: 0.0224, Acc: 0.9922 | Val Loss: 0.4557, Acc: 0.9182
Epoch 25/30 | Train Loss: 0.0193, Acc: 0.9933 | Val Loss: 0.5591, Acc: 0.9120
Epoch 30/30 | Train Loss: 0.0157, Acc: 0.9949 | Val Loss: 0.6026, Acc: 0.9149
Test Accuracy: 0.9141
```

```
=====
Kernel size: 5x5
=====
Epoch 5/30 | Train Loss: 0.1709, Acc: 0.9360 | Val Loss: 0.2383, Acc: 0.9144
Epoch 10/30 | Train Loss: 0.0736, Acc: 0.9727 | Val Loss: 0.3155, Acc: 0.9152
Epoch 15/30 | Train Loss: 0.0371, Acc: 0.9863 | Val Loss: 0.4259, Acc: 0.9192
Epoch 20/30 | Train Loss: 0.0268, Acc: 0.9902 | Val Loss: 0.5175, Acc: 0.9119
Epoch 25/30 | Train Loss: 0.0198, Acc: 0.9931 | Val Loss: 0.5942, Acc: 0.9131
Epoch 30/30 | Train Loss: 0.0189, Acc: 0.9936 | Val Loss: 0.6514, Acc: 0.9138
Test Accuracy: 0.9137
```

```
=====
Kernel size: 7x7
=====
Epoch 5/30 | Train Loss: 0.1792, Acc: 0.9327 | Val Loss: 0.2670, Acc: 0.9040
Epoch 10/30 | Train Loss: 0.0783, Acc: 0.9706 | Val Loss: 0.3263, Acc: 0.9121
Epoch 15/30 | Train Loss: 0.0457, Acc: 0.9833 | Val Loss: 0.4415, Acc: 0.9036
```

Epoch 20/30 | Train Loss: 0.0290, Acc: 0.9893 | Val Loss: 0.5487, Acc: 0.9099
Epoch 25/30 | Train Loss: 0.0270, Acc: 0.9907 | Val Loss: 0.6244, Acc: 0.9113
Epoch 30/30 | Train Loss: 0.0259, Acc: 0.9915 | Val Loss: 0.7183, Acc: 0.9056
Test Accuracy: 0.9116

池化方式（最大池化 vs 平均池化）实验终端输出：

=====

Pooling type: Max Pooling

=====

Epoch 5/30 | Train Loss: 0.1776, Acc: 0.9352 | Val Loss: 0.2521, Acc: 0.9082
Epoch 10/30 | Train Loss: 0.0797, Acc: 0.9703 | Val Loss: 0.2568, Acc: 0.9209
Epoch 15/30 | Train Loss: 0.0365, Acc: 0.9868 | Val Loss: 0.3620, Acc: 0.9179
Epoch 20/30 | Train Loss: 0.0224, Acc: 0.9922 | Val Loss: 0.4557, Acc: 0.9182
Epoch 25/30 | Train Loss: 0.0193, Acc: 0.9933 | Val Loss: 0.5591, Acc: 0.9120
Epoch 30/30 | Train Loss: 0.0157, Acc: 0.9949 | Val Loss: 0.6026, Acc: 0.9149
Test Accuracy: 0.9141

=====

Pooling type: Average Pooling

=====

Epoch 5/30 | Train Loss: 0.2253, Acc: 0.9178 | Val Loss: 0.2666, Acc: 0.9037
Epoch 10/30 | Train Loss: 0.1460, Acc: 0.9457 | Val Loss: 0.2148, Acc: 0.9219
Epoch 15/30 | Train Loss: 0.0900, Acc: 0.9669 | Val Loss: 0.2387, Acc: 0.9246
Epoch 20/30 | Train Loss: 0.0519, Acc: 0.9811 | Val Loss: 0.3057, Acc: 0.9234
Epoch 25/30 | Train Loss: 0.0319, Acc: 0.9889 | Val Loss: 0.3969, Acc: 0.9163
Epoch 30/30 | Train Loss: 0.0237, Acc: 0.9921 | Val Loss: 0.4316, Acc: 0.9189
Test Accuracy: 0.9230